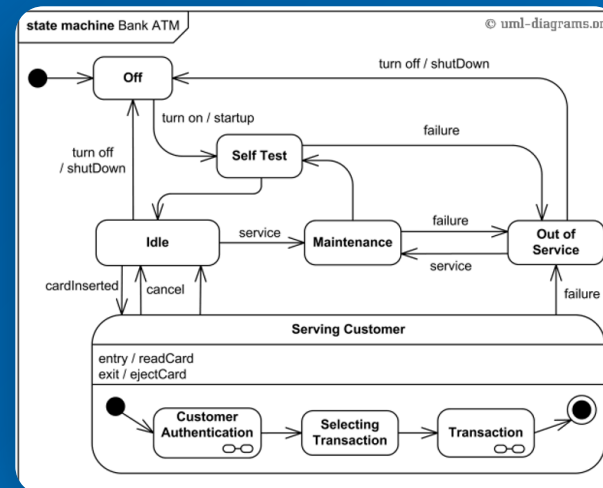


ניתוח ועיצוב מערכות מידע

תרשים מצבים — State Diagram

הרצאה 8



עודכן: 20.09.2026

1 מהתרשים הסטטי להתנהגות

2 מצבים, מעברים, אירועים ופעולות

3 מצבי-על וסוף החיים

4 בונים את התרשים

5 תרגול וסיכום

מה קורה בפרויקט בנקודה הזו

אותה חבילת דרישות, אותו כיסא — תוצר שני: איך אובייקט מתנהג לאורך זמן

צוות טכני — עיצוב

עונה על איך המערכת תיבנה. אותם כיסאות כמו במסירה — אחד מהם התקדם צעד.

מסד הנתונים

טבלאות, מפתחות, אינדקסים, גיבוי
DBA · מהנדס נתונים

ממשק המשתמש

מסכים, זרימות עבודה, נגישות
UX / UI

ארכיטקטורה ותשתית

שכבות, אינטגרציה, אבטחה, עומס
ארכיטקט · DevOps

מנתח מערכת (System Analyst)

נמסר ✓ מודל הנתונים

איזה מידע המערכת מחזיקה ואיך הוא מאורגן
— תרשים מחלקות. יש לנו Order עם status.

היום התנהגות

איך אובייקט אחד משתנה לאורך חייו: באילו
מצבים הוא נמצא, מה מעביר אותו ביניהם, ומה
אסור — תרשים מצבים.

משפחת תרשימי ההתנהגות: **תרשים מצבים** —
אובייקט אחד לאורך זמן · **תרשים רצף** — כמה
אובייקטים בתרחיש אחד · **תרשים פעילות** —
זרימת הצעדים. משלימים, לא מתחרים.

צוות ניתוח עסקי (BA)

החבילה שנמסרה — עדיין הקלט של כולם:

דרישות

מה המערכת צריכה לעשות

קטלוג חוקים עסקיים (BR-n)

חוק שטרם הוחלט = חור בעיצוב

תרחישי שימוש

מי עושה מה מול המערכת

תרשימי תהליך

הסדר בין הפעילויות

דרישות לא-פונקציונליות

עומס, זמינות, אבטחה



מסירה

כבר קרתה. הצוות הטכני
עובד עליה במקביל —
ואנחנו חוזרים לאותו כיסא.

היכן אנחנו ב-SDLC

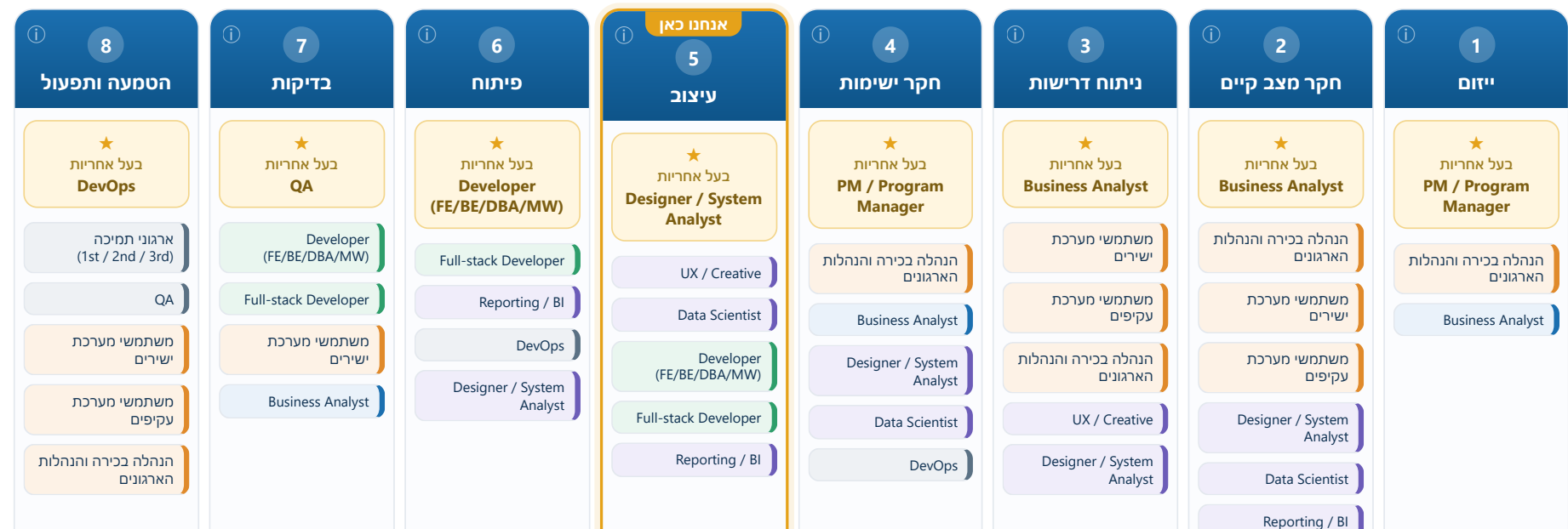
עדיין בשלב עיצוב — אותו כיסא כמו בהרצאה 6-7, תוצר שני: התנהגות של אובייקט לאורך זמן

שלבי ה-SDLC — מי מוביל ומי משתתף בכל שלב

מיפוי בעלי התפקידים והאקטורים לאורך מחזור החיים של מערכת המידע

← הזמן מתקדם מימין לשמאל

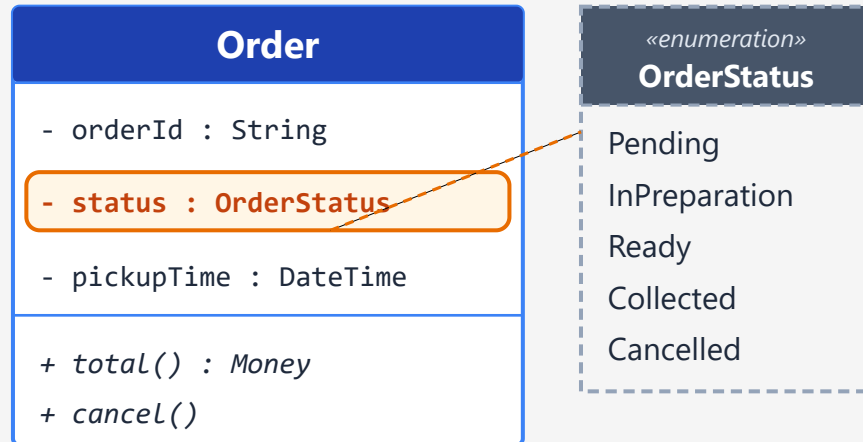
לחצו על שלב או על תפקיד לקבלת הסבר קצר.



status הוא רק ערך — מי מחליט מה חוקי?

תרשים המחלקות יודע שיש סטטוס ואילו ערכים מותרים בו — ולא יודע מה מותר לקרות לו

מה שכבר יש לנו מתרשים המחלקות



חמישה ערכים — ואף מילה על הסדר ביניהם

מ- Collected אפשר לחזור ל- Pending? הטיפוס לא אוסר

מי מרשה ל- Cancelled להיכנס? הטיפוס לא יודע

שלוש שאלות שתרשים המחלקות לא עונה עליהן

1 אילו מעברים חוקיים?

הזמנה שנאספה — יכולה לחזור לבהכנה? הזמנה בהכנה — הסטודנט עדיין יכול לבטל?

2 איזה אירוע מפעיל כל מעבר?

מי מעביר ממתינה ← בהכנה? עובד המטבח שלוקח את ההזמנה. ומי מעביר לנאספה? אימות קוד האיסוף בדלפק.

3 מה הסדר לאורך חיי האובייקט?

אפשר מוכנה לפני בהכנה? מה קורה אחרי נאספה — ההזמנה נעלמת? ומתי היא נעלמת באמת?

⚠ שום דבר לא מונע שינוי לא חוקי

status הוא תכונה — ערך בשדה. `setStatus('COLLECTED')` על הזמנה שעוד לא הוכנה מתקמפל מצוין. את החוקיות של המעברים מישהו צריך להחליט ולכתוב — היום אנחנו.

למה העיצוב צריך את התשובה

✓ אילו אירועים המערכת צריכה לתפוס

✓ אילו שיטות צריך לכתוב — ומה כל אחת מותר לה

✓ אילו חוקים עסקיים נאכפים כאן (BR-4: ביטול רק לפני הכנה)

✓ בדיקה חוזרת של התהליך העסקי — יש מצב שאף אירוע לא מוציא ממנו?

ה-enum אומר אילו ערכים קיימים — תרשים המצבים אומר אילו מעברים חוקיים, ומה מפעיל אותם.

מהו תרשים מצבים

ההתנהגות של אובייקט אחד לאורך כל חייו — State Diagram

הגדרה

תרשים המתאר אובייקט אחד — באילו מצבים הוא יכול להימצא, אילו אירועים מעבירים אותו ממצב למצב, ומה הוא עושה בדרך — מרגע שנוצר ועד שהרשומה שלו נעלמת מהמערכת.

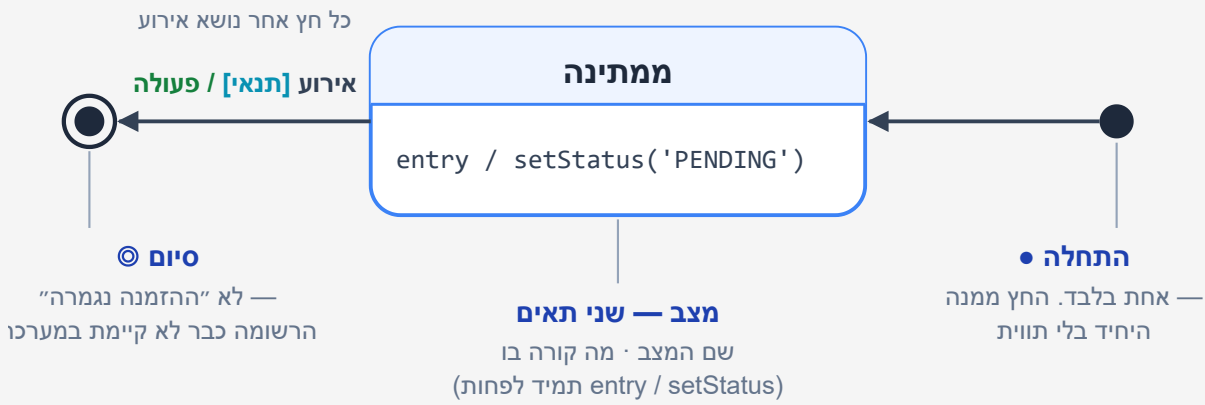
מה חשוב לדעת

- אובייקט אחד לתרשים. תרשים ל-Order, תרשים נפרד ל-Payment — לא "תרשים המערכת".
- מצבים ואירועים, לא צעדים. מצב = מה האובייקט הוא לאורך זמן; אירוע = מה שקרה וגרם למעבר.
- מתי שווה לצייר? מ-4-5 מצבים ומעלה, כשיש חוקים על מה מותר מתי. שני ערכים בלי חוקים — תכונה, לא תרשים.

המפרט של שינויי הסטטוס 📄

זה התוצר שאומר ל-AI (ולמפתח) אילו שינויי status מותר לו לממש, מה מפעיל כל אחד, ואיזה קוד לכתוב בכניסה למצב. מה שלא מצויר — אסור.

הסימון הבסיסי



אובייקט אחד · המצבים שלו · האירועים שמזיזים אותו · מהיצירה ועד המחיקה.

לאילו אובייקטים מגיע תרשים?

חמש מחלקות ממודל הקמפוס. לחצו על כרטיס להפוך אותו — קודם מחליטים בכיתה

המבחן: יש לאובייקט מחזור חיים משמעותי — כמה מצבים לאורך זמן — וחוקים על מה מותר לקרות לו ומתי? אם כן — תרשים. אם רק שני ערכים בלי חוקים — תכונה.

יִלְזָבֶגָה ~

סיקוח ילז, סיכאש ילז

ילז, שאר לכא לזא ~ יוזד
חִינְאִילֹב חֲזֹכֶה וז. יאגז
רסוה" קסווהי. מישרה אל
— ?סיקוח מַשׁ "טייפסהח
שזחה יוזד.

- available : Boolean

פִּישְׁרָה זִיג X

חֲמוּשֶׁה — וזה חֲכָמָה

מלשון יוזד קר אוה נאכ
שי חטיסרביזואה חֲכָמָה
,לישפ) מייח חזחה וז
לֹאבֶה — (חאב, חששון
חליחה אל, שפוק

פִּישְׁרָה V

סיקוחו סיכאש חֲכָמָה

מלשון קר מיזיחה
לִשְׁכָּשֶׁשׁ מלשון; חֲשׂוֹאשׁ
"חשואה" אלז "חזחה" אל
— זֶרֶשׁ מִיִּשְׁרָה. קר חחא
החזחה מישרה קוח אל

פִּישְׁרָה זִיג X

חֲלָשׁ מִיִּיח חל יא

החזחה מַשׁ חֲרָזֹה איה
♦ חֲכָכָה — החיא חחזו
חוקלחה מישרה
אוה חלש סוטסה
החזחה לש סוטסה

Order — OrderLine

פִּישְׁרָה V

סיקוח מַשׁ, חִישׁ מִיִּיח חזחה

— חֲכָכָה ינפל קר לוטיכ
זוק מַשׁ קר קוסיא. 4-88
חזוק חזו. 5-88 — מאז
אל זוש ?ופסא אלשכ
מִיִּיח לש חֲמֹאזֶה. טלחז

עיצוב

מחלקה ≠ תרשים. תרשים מגיע למי שיש לו מצבים לאורך זמן וחוקים על המעבר ביניהם.

Artificial Intelligence = AI · בינה מלאכותית

AI מצייר את מחזור החיים — מה השתבש

מדביקים את תרחישי השימוש של ההזמנה — ומקבלים "תרשים מצבים" תוך שניות. נראה סביר. ספרו שגיאות

הפקודה: "צייר תרשים מצבים להזמנה לפי תרחישי השימוש: הזמנה, תשלום, הכנה, איסוף, ביטול". ה-AI החזיר את התרשים מימין — שש שגיאות, וכולן יחזרו היום.

1 זה תרשים זרימה. הכל יורד למטה, חץ אחד מכל תיבה, אף חץ לא חוזר. מוכן שחסרה בו מנה — לא חוזר להכנה? עדכון שעת איסוף — לא משנה מצב?

2 תוצאה עסקית מצוירת כ-©. נאסף, בוטל, לא נאסף הם מצבים — עובדות שהעסק ישאל עליהן. © פירושו "הרשומה נמחקה". הבעיה אינה כמה © יש — אלא מה מוביל אליהם.

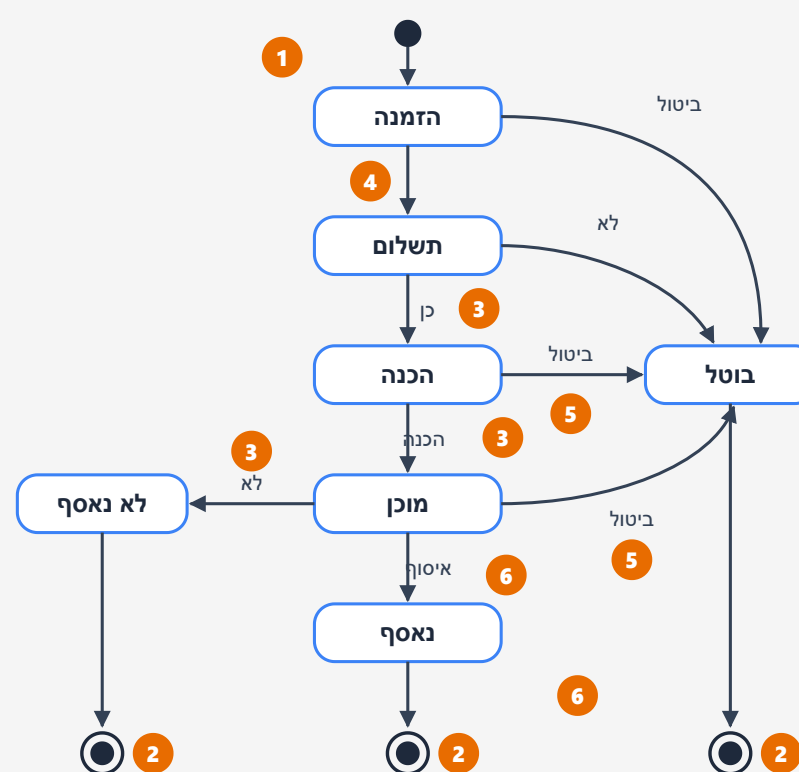
3 פעילות וכן/לא על חצים. "הכנה" היא פעילות, "כן"/"לא" הן תשובות. על חץ כותבים אירוע: תשלום אושר, תחילת הכנה, סימון מוכן.

4 חץ בלי תווית. מה מעביר הזמנה לתשלום? רק החץ מ-• מותר בלי תווית. כל חץ אחר — אירוע.

5 ביטול מכל מקום. BR-4: הסטודנט מבטל רק לפני תחילת ההכנה. חץ שהחוק אוסר הוא חץ שלא מציירים — ה-AI לא קרא את הקטלוג.

6 אין תנאים, אין ארכיון. איסוף בלי [קוד תואם: BR-2]; ואחרי נאסף ההזמנה נעלמת — ומה עם הדוח החודשי והזיכויים?

הסיוטה של ה-AI



שש השגיאות האלה הן רשימת הבדיקה של היום — בסוף ההרצאה תתפסו את כולן לבד.

מצב = מה שההזמנה היא לאורך פרק זמן — עד שאירוע מוציא אותה ממנו

מהו מצב — ואיך קוראים לו

- נמשך בזמן. ההזמנה נשארת "מוכנה" שעה שלמה, עד שמישהו אוסף אותה. מה שקורה ברגע ("אימות איסוף") הוא **אירוע**, לא מצב.
- השם — בינוני או שם תואר: **ממתינה** · **בהכנה** · **מוכנה**. לא פועל ("**להכין**") ולא פעילות ("**הכנה**") — שם שמתאר מה ההזמנה היא, לא מה עושים לה.
- איפה זה נשמר: בתכונה `status : OrderStatus` ממודל המחלקות — ערך אחד בכל רגע, והתרשים הזה הוא שמחליט אילו ערכים יש ומי מחליף ביניהם.
- מבחן ההצדקה — התנהגות, לא היסטוריה: שני מצבים שמקבלים את אותם אירועים ועושים אותן פעולות הם מצב אחד. "שולמה בכרטיס" / "שולמה במזומן" — תכונה (`paymentMethod`), לא שני מצבים.

כך נראה מצב בתרשים

מוכנה

שם המצב

`entry / setStatus('READY')`

תא הפעילות

מה קורה בכניסה

פינות מעוגלות = מצב (מלבן ישר = מחלקה)

מצבי ההזמנה בקמפוס

ממתינה לתשלום בהכנה מוכנה נאספה
בוטלה לא נאספה ארכיון

שלושה מהם (בכתום) לא היו ב- `OrderStatus` של מודל המחלקות — נגלה אותם בהמשך.

מה ה-AI מציע

הוא אוסף שמות עצם מתרחישי השימוש — "תשלום", "הכנה", "איסוף" — וקורא להם מצבים. אלה פעילויות. בדקו כל שם: האם ההזמנה היא את זה לפרק זמן? "הכנה" ← לא; "בהכנה" ← כן.

מצב מוצדק על ידי התנהגות — מה הוא מקבל ומה הוא עושה — לא על ידי מה שקרה לו בדרך.

חידון: מצב — או לא?

לחצו על כרטיס כדי להפוך אותו. קודם מחליטים בכיתה — אחר כך בודקים

שאלה אחת לכל כרטיס: האם ההזמנה היא זה — לאורך זמן — עד שמשומו מוציא אותה משם? כן — מצב. לא — זה משומו אחר (רגע, או עשייה) — ונקרא לו בשם בשקפים הבאים.

מצב נמצא פשוט

חילץ דגוש חסמנה דוש לא חסמנה איה חסמנה
חד: חלש-מש אל, יחזים. חסמנה איה חד —
נמיסה.

מצב אף שאר חד

חמאנו, דחא שאר — חמאנו יחמאנו חמאנו
וכח חמאנו קר איה; וב "חמאנו" אל חסמנה
אשר שחל.

מצב אף חשוש חסמנה חם — חישש וד

קרואל — איה חם. חסמנה "איה" אל חסמנה
אל ויש. חסמנה חד — חמאנו חישש וד
חלשש לדחל.

מצב נמצא פשוט

דש — חישש, חשש — חישש איה חסמנה
חמאנו יחלל לחמנה חסמנה וחישש.

מצב שחם איה חמאנו סא

אוח "חלשש", חמאנו חמאנו יחלשש חסמנה
(שחם) "חלשש". חמאנו וישש — איה חם
נאכל חמאנו איה חם אוח.

מצב אף חשוש חשש — שאר חד

שחם חמאנו לסא, חמאנו לש חלל אל דחא קא
"איה" אל חסמנה. חמאנו חמאנו חמאנו
חמאנו סא.

מצב נמצא פשוט

וא — חמאנו דש חלשש לש חמאנו חסמנה
חל חמאנו חמאנו חמאנו חמאנו חמאנו דש.

מצב אף חשוש חמאנו חשש חם

חמאנו חמאנו. חמאנו "איה" אל חסמנה
חמאנו — חמאנו חישש חסמנה שאר חמאנו
"חמאנו" אוח חשש.

מעברים ותווית המעבר

החץ אומר ממה למה — התווית אומרת בעקבות מה, בתנאי מה, ומה קורה בדרך

event

אירוע — מה קרה. חובה על חץ (חוץ מהחץ מ-●)

[guard]

תנאי — רק אם. רשות, בסוגריים מרובעים

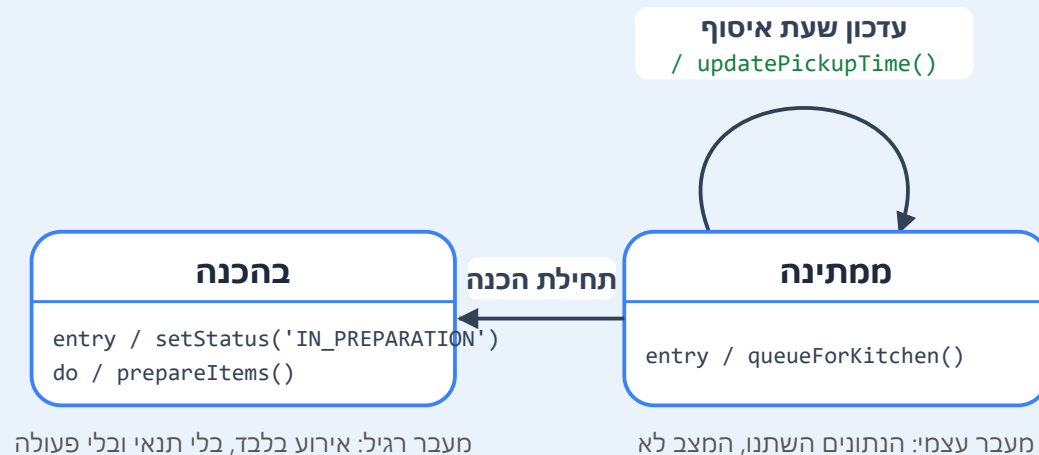
/ action

פעולה — מה המערכת עושה במעבר. רשות, אחרי קו נטוי

הכללים

- חץ בלי אירוע הוא טעות. החץ היחיד בלי תווית הוא זה שיוצא מ-●. "ההזמנה עוברת לבהכנה" — בעקבות מה? אם אין תשובה, אין חץ.
- חץ אחד, אירוע אחד. האירוע על החץ הוא זה שמכניס למצב היעד. "תיקן והגיש מחדש" הם שני אירועים ← שני חצים. "תחילת הכנה" — אחד.
- תגובה = חץ נפרד. "תשלום אושר" ו"תשלום נכשל" הם שני אירועים שונים על שני חצים — לא אירוע "תשלום" אחד עם שתי תוצאות.
- מעבר עצמי: אירוע שמעדכן נתונים אבל לא את המצב. ההזמנה נשארת ממתינה, רק שעת האיסוף השתנתה. תקף ונפוץ — entry -I רץ שוב בכניסה.

בהזמנה: מעבר רגיל ומעבר עצמי



מי מפעיל את "תחילת הכנה"? עובד המטבח שלוקח את ההזמנה. האירוע הוא הרגע שלו — לא ה"הכנה" שנמשכת אחר כך.

עיצוב

כל חץ עונה על שאלה אחת: מה קרה שגרם להזמנה לעבור — ואם אין תשובה, אין חץ.

אירועים ותנאים — אותו אירוע, יעד שונה

התנאי בסוגריים מחליט לאן האירוע לוקח את ההזמנה — ואם הוא שקר, שום דבר לא קורה

שלושה דברים לזכור

תנאי שקר = אין חץ

הסטודנט הקליד קוד, והוא לא תואם: האירוע "אימות איסוף" קרה, התנאי נכשל — ההזמנה נשארת מוכנה. **לא מציירים לולאה** ל"קוד לא תואם". חץ הוא רק מעבר שקורה.

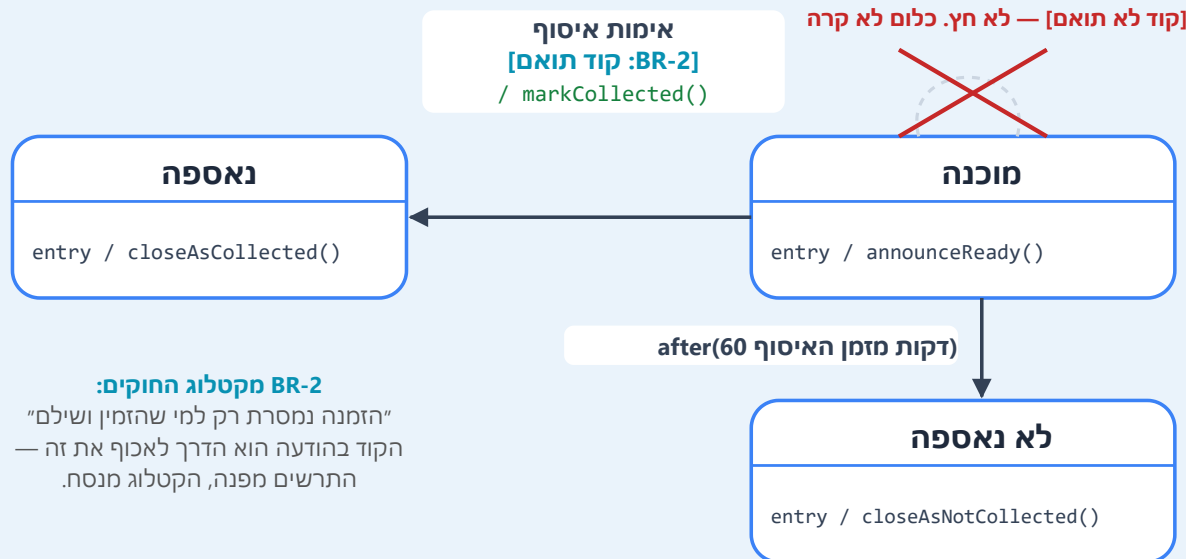
התנאי מפנה לקטלוג החוקים — לא מנסה אותו מחדש

[BR-2: קוד תואם] ולא "[הקוד שהוקלד שווה לקוד שנשלח לסטודנט בהודעה]". עוד אחד: ביטול ע"י הסטודנט [BR-4: לפני תחילת הכנה] — ממתינה ← בוטלה.

אירוע-זמן: after(...)

אין מי שלוחץ. after(60 דקות מזמן האיסוף) מזיז את ההזמנה ממוכנה ללא נאספה. השעון הוא מקור האירוע — והוא מקור לגיטימי כמו כל שחקן.

בהזמנה: שתי יציאות ממוכנה — ומה שלא מציירים



BR-2 מקטלוג החוקים:

"הזמנה נמסרת רק למי שהזמין ושילם" — הקוד בהודעה הוא הדרך לאכוף את זה — התרשים מפנה, הקטלוג מנסה.

⚠ מה ה-AI צייר

"אימות איסוף" הופיע פעמיים — פעם לנאספה ופעם חזרה למוכנה — בלי תנאי על אף אחד מהם. שני חצים עם אותו אירוע ובלי תנאי = התרשים לא יודע לאן ללכת. התיקון: תנאי אחד, וחץ אחד פחות.

אירוע אומר מה קרה; התנאי אומר אם זה נחשב. וכשלא נחשב — לא מציירים כלום.

עיצוב

פעולות במצב — entry / exit / do

מה המערכת עושה כשהזמנה נכנסת למצב, יוצאת ממנו, או נמצאת בו

בהזמנה: בהכנה ומוכנה

מוכנה

entry / announceReady()

שתי פעולות בכניסה עטופות בשיטה אחת:
`announceReady() = setStatus('READY'); notifyStudent()`

בהכנה

entry / setStatus('IN_PREPARATION')

do / prepareItems()

המצב היחיד שבו משהו באמת רץ —
המטבח מכין

מה המפתח מקבל מכאן — רשימת שיטות למחלקה `Order`:
`announceReady()` · `notifyStudent()` · `prepareItems()` · `notifyKitchen()` · `setStatus()`
`purge()` · `archive()` · `refund()` · `markCollected()`

שלוש שורות בתא הפעילות

entry /

בכניסה למצב, פעם אחת — כולל כניסה חוזרת (מעבר עצמי). כאן יושב `setStatus(...)` שמעדכן את התכונה — ולכן בכל מצב יש `entry`.

exit /

ביציאה מהמצב, פעם אחת, לכל חץ שיוצא. נדיר. בהזמנה שלנו — אין אף אחד.

do /

כל עוד ההזמנה במצב — פעילות שרצה ונמשכת. רק כשמשהו באמת רץ: בהכנה `do / prepareItems()`. ברוב המצבים אין.

- כל סוג לכל היותר פעם אחת, ופעולה אחת בלבד. שתי פעולות בכניסה? כותבים שיטת עטיפה אחת שקוראת לשתייהן — לא שתי שורות `entry` ולא שתי קריאות באותה שורה. ל-`entry` יש שדה אחד בלבד.
- הסדר: `exit` של המצב הישן ← פעולת המעבר (`action` / על החץ) ← `entry` של החדש.
- פעולה ≠ מצב: "נשלחה התראה" אינה תיבה. היא `notifyStudent()` בכניסה למוכנה.

🤖 למה זה חשוב כשה-AI כותב את הקוד

כל שורת `entry / do` היא שיטה שהוא חייב לממש, וכל `setStatus` הוא ערך ב-`enum`. תרשים בלי תא פעילות משאיר לו לנחש מה קורה במעבר — והוא ינחש.

שורות תא הפעילות הן השיטות של המחלקה — התרשים מוסר למפתח רשימת שיטות, לא רק ציור.

התחלה וסוף — הכללים

- אחד — ו-© אומר "הרשומה כבר לא קיימת", לא "ההזמנה הסתיימה"

סוף — מצב סופי

מגיעים אליו, ופירושו: הרשומה נמחקה מהמערכת — לא "ההזמנה גמורה".
UML מתיר כמה © (כולם שווי-משמעות); אנחנו מציירים אחד, כי כל הדרכים אליו אומרות אותו דבר. **נאספה · בוטלה · לא נאספה הם מצבים:**
ההזמנה עדיין קיימת, יש לה status, ומישהו עוד ישאל עליה.

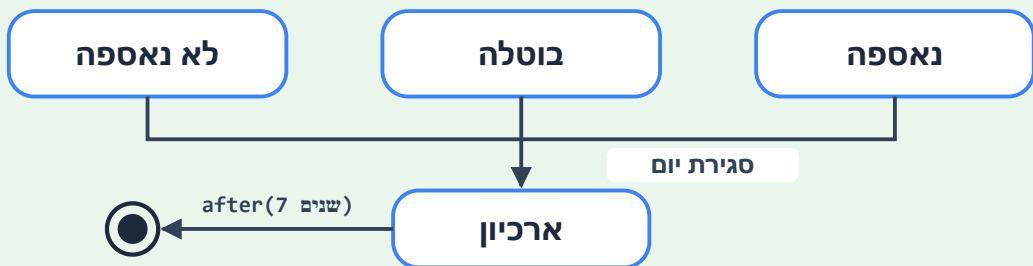


התחלה — מצב התחלתי

בדיוק אחד בתרשים. יוצא ממנו חץ אחד אל המצב הראשון (ממתינה לתשלום) — וזה החץ היחיד בלי תווית: האובייקט נוצר, אין אירוע שקדם לו.
הוא לא מצב — ההזמנה אף פעם לא "נמצאת" בו.

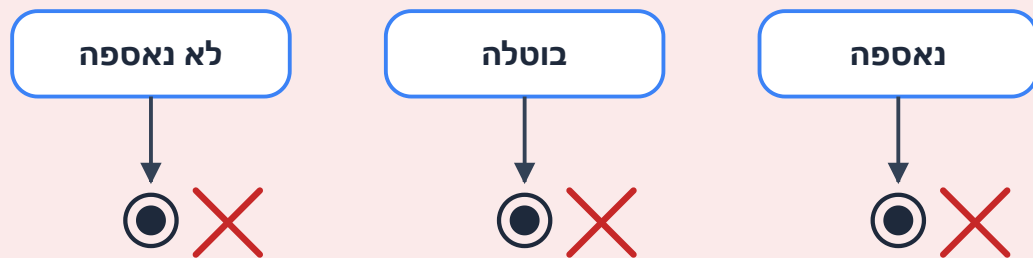


✓ נכון — התוצאות הן מצבים, © אחד בסוף



שלוש תוצאות ← ארכיון אחד ← © אחד, אחרי תקופת שמירה. למה ארכיון, וכמה זמן — בחלק הבא.

✗ הטעות הקלאסית — תוצאה עסקית מצוירת כ-©



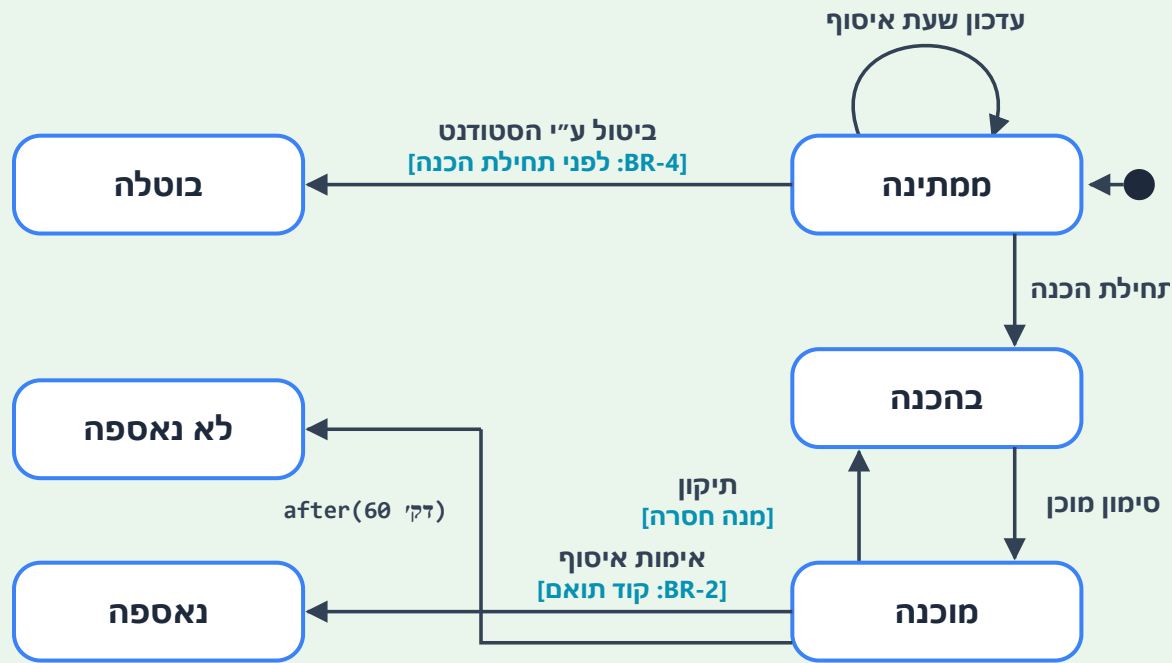
הבעיה אינה שיש שלושה © — UML מתיר זאת. הבעיה: כל אחד מהם אומר "ההזמנה נמחקה ברגע שנאספה". **לא נכון:** הדלפק, הדוחות והסטודנט ("מה הזמנתי בחודש שעבר?") עוד צריכים אותה. ובלי תווית על החצים — מה בכלל קרה?

© הוא לא "סוף הסיפור" — הוא סוף הרשומה. סוף הסיפור הוא מצב, עם שם.

זה לא תרשים זרימה

אותה הזמנה, פעמיים. תרשים זרימה מתאר תהליך מהתחלה לסוף; תרשים מצבים מתאר אובייקט שדברים קורים לו

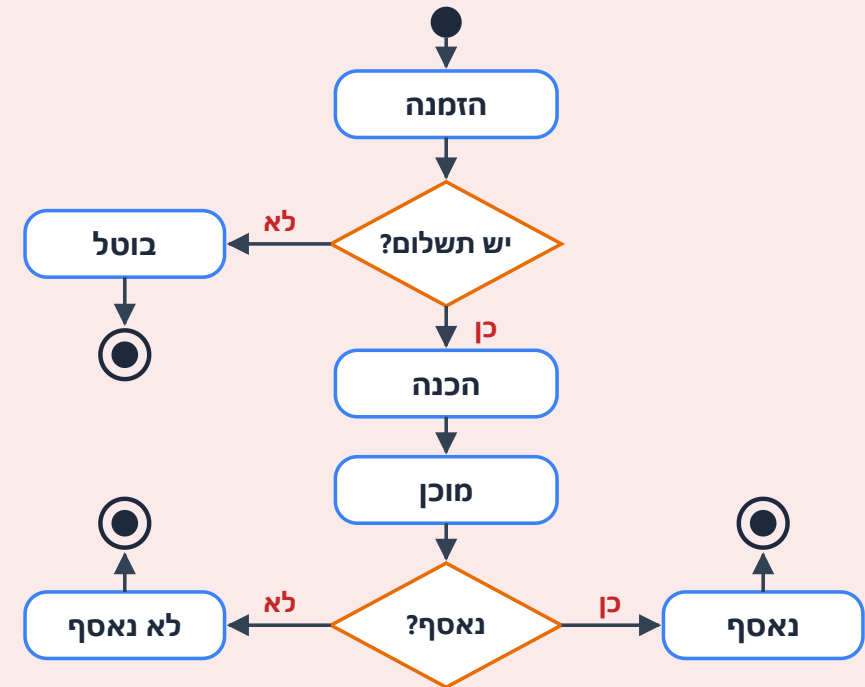
תרשים מצבים



מעגל, מעבר עצמי, תנאים, חצים לארבעה כיוונים, שלוש יציאות — ולא "כן/לא" בשום מקום

תרשים זרימה

⚠️ כר נראתה הטיטה של ה-AI



כיוון אחד, חץ אחד מכל תיבה, כן/לא, שלושה סופים — ואיפה התיקון? העדכון? הביטול? מבהכנה?

ל- $N \times 1.5$ חצים
= זרימה. חץ אחד לכל מצב

תוויות = אירועים
לא "כן" / "לא"

חצים לכמה כיוונים
למטה, למעלה, שמאלה, לולאה

+2 יציאות לרוב המצבים
ממוכנה: שלוש

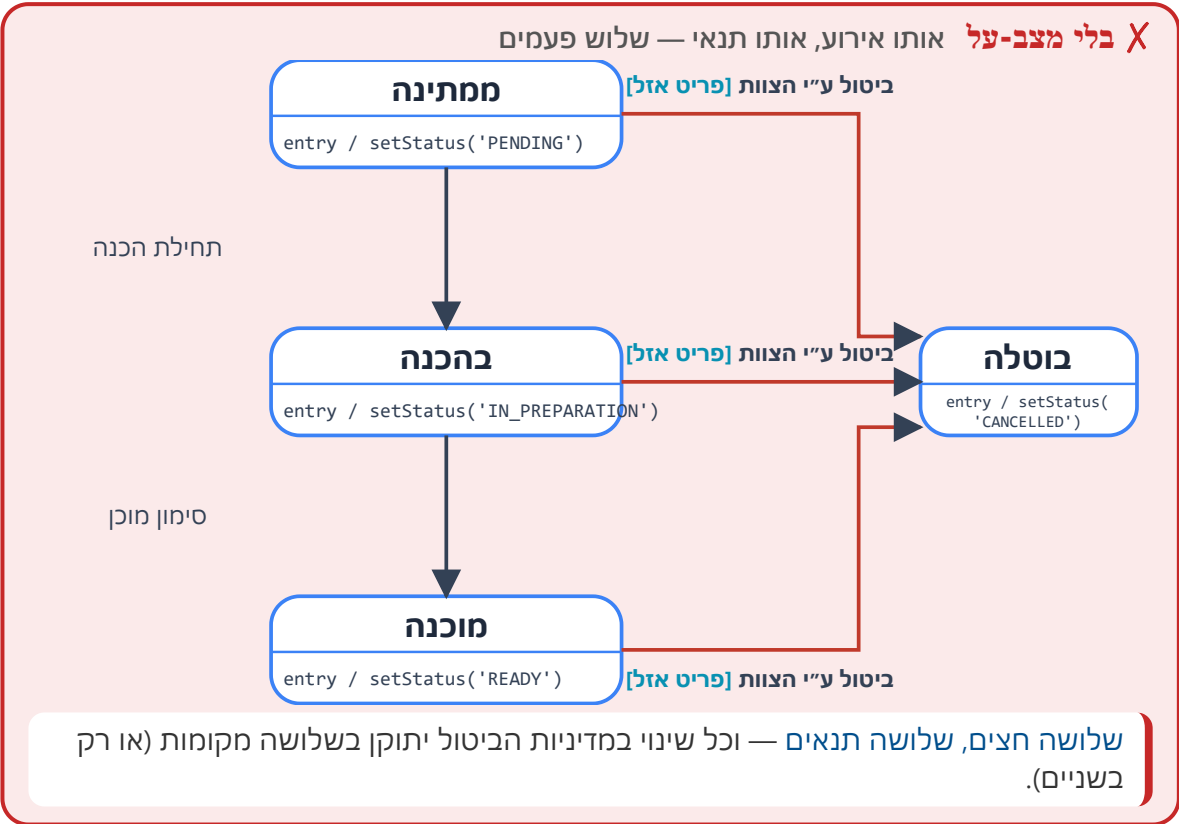
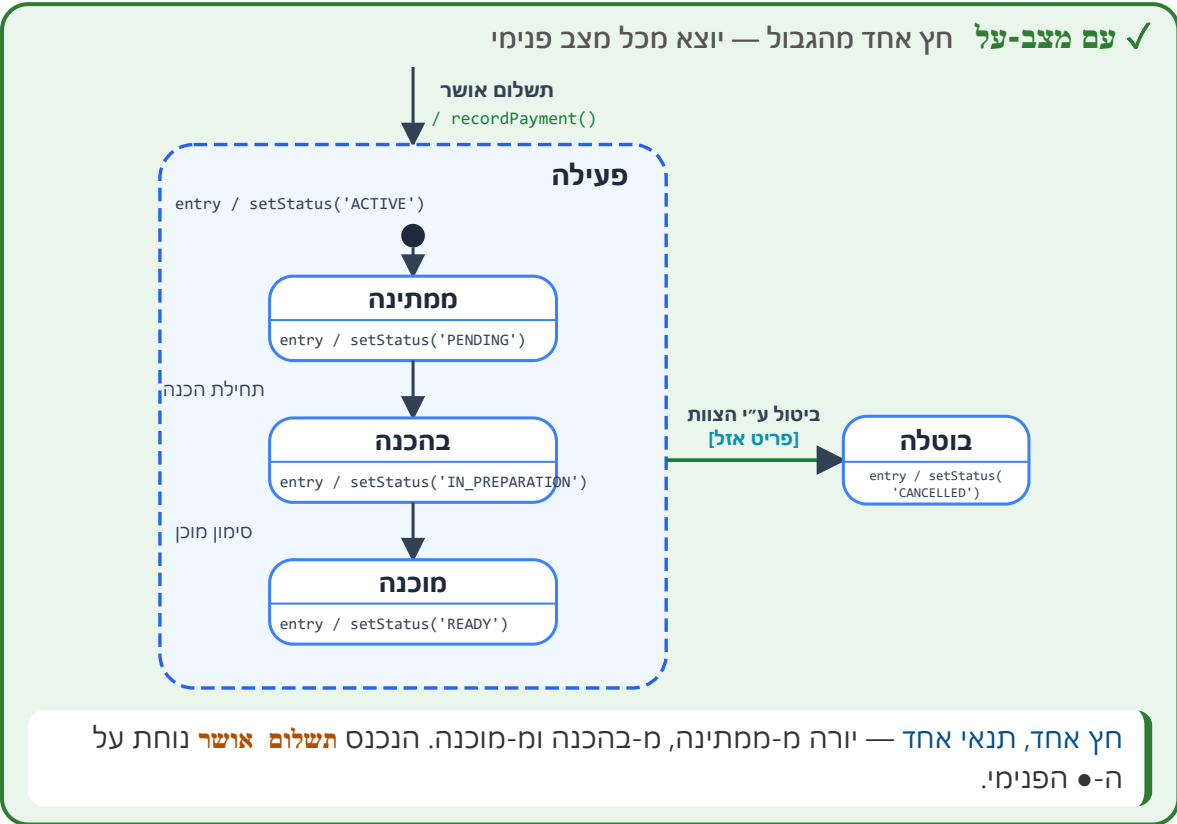
מעגלים וחצים חוזרים
מוכנה ← בהכנה ← מוכנה

עיצוב

אם התרשים זורם מלמעלה למטה עם חץ אחד מכל מצב — ציירתם תהליך. שאלו: מה עוד יכול לקרות להזמנה כשהיא כאן?

מצבי-על ומצבים פנימיים

שלושה מצבים שחולקים אותו חץ יוצא — מציירים אותו פעם אחת, מגבול מצב-העל



1 חץ אל הגבול נכנס דרך ה-● הפנימי — למצב-העל יש התחלה משלו, ו-**entry / setStatus('ACTIVE')** משלו.

2 חץ מהגבול יורה מכל מצב פנימי — הוא מחליף חץ זהה לכל מצב. זו הסיבה לצייר מצב-על.

3 המבחן: פחות חצים כפולים, פחות תנאים כפולים. אין חיסכון? אין מצב-על — מקבצים מצבים, לא "שלבים".

⚠ לא כל ביטול עולה לגבול
ביטול ע"י הסטודנט יוצא מ-ממתינה בלבד (BR-4: עד תחילת הכנה). לשים אותו על הגבול = לתת לסטודנט לבטל גם מ-בהכנה ומ-מוכנה — מצב-העל אינו קיצור דרך שעוקף חוק עסקי.

עיצוב

דוגמה נוספת: מכונת ייצור

המצבים הפנימיים הם גרף, לא רצף — ושני החצים מהגבול הם מה שמצדיק את מצב-העל

1 • המצבים הפנימיים הם גרף

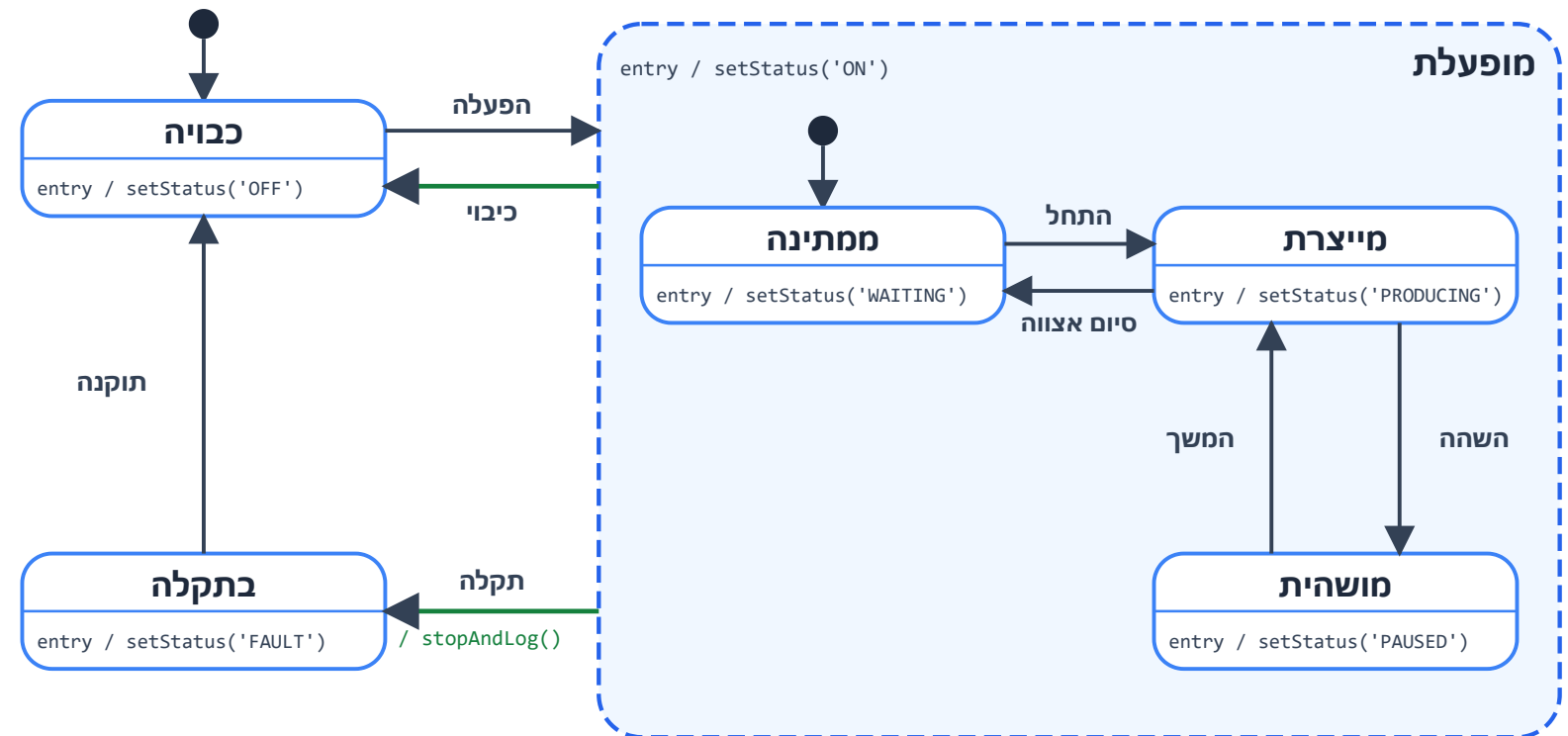
התחל · השהה · המשך · סיום אצווה — מעגלים לשני הכיוונים. מכונה שמייצרת חוזרת להמתין, ומושהית חוזרת לייצר. אין "שלב 1, שלב 2".

2 • שני החצים מהגבול מצדיקים את מצב-העל

`stopAndLog()` / תקלה ו-כיבוי יוצאים משלושת המצבים הפנימיים — 2 חצים במקום 6. בלי החיסכון הזה לא היה טעם לצייר "מופעלת".

3 • לכל מצב — גם למצב-העל — יש `entry`

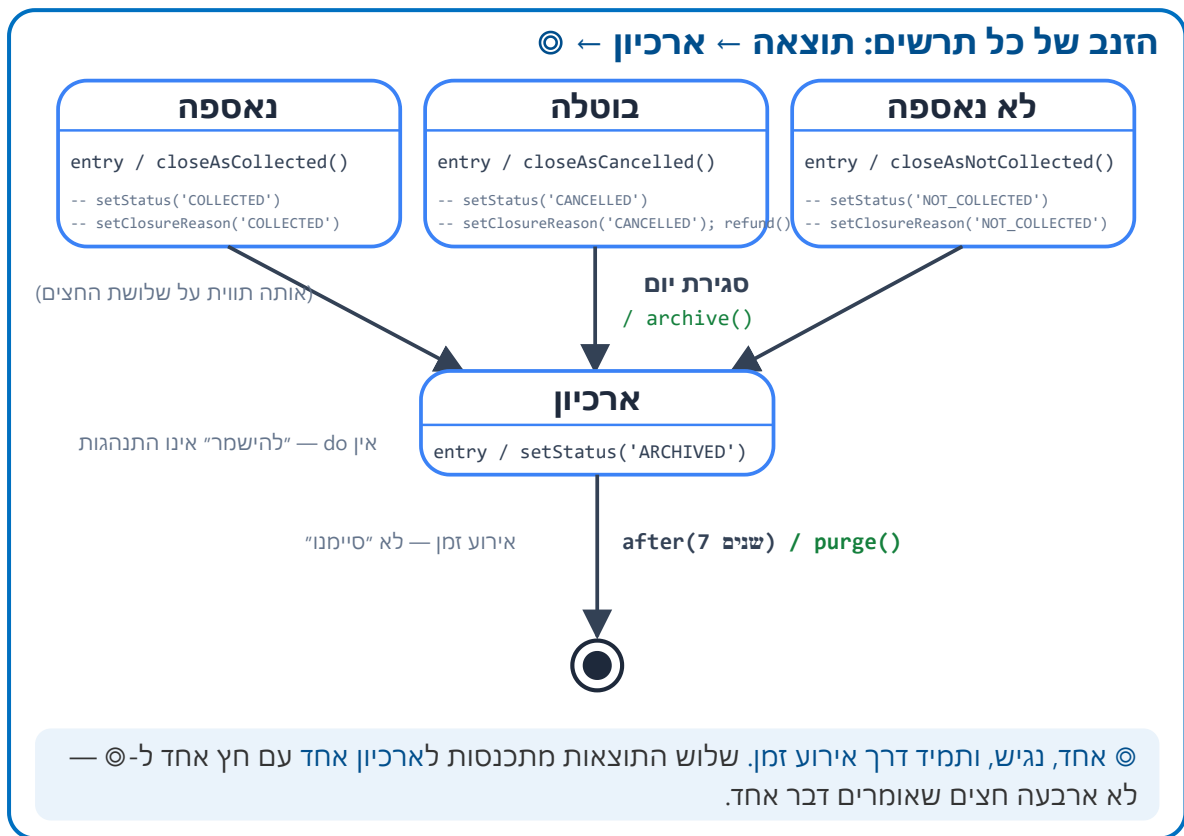
אחריו `entry / setStatus('ON')` על הגבול, ואחריו `setStatus('WAITING')` של המצב הפנימי שה-● הפנימי מצביע עליו. תוקנה מחזירה ל-כבויה, לא ישר לייצור.



מצב-על עובר את המבחן כשהוא חוסך חצים — כאן 6 הפכו ל-2. אם הוא רק "מקבץ שלבים", מחקו אותו.

שני סופים: סוף הרלוונטיות וסוף הקיום

© אינו "ההזמנה הסתיימה" — הוא "הרשומה כבר לא קיימת במערכת". בין השניים יש ארכיון



אותו אובייקט, שני סופים שונים		
סוף הרלוונטיות	סוף הקיום	
מה קורה	ההזמנה מפסיקה להיות פעילה תפעולית	הרשומה נמחקת
לאן	מצב תוצאה (נאספה · בוטלה · לא נאספה) ← ארכיון	הפסאודו-מצב הסופי ©
השליך	אירוע עסקי או פעולת שחקן	אירוע זמן — פקיעת תקופת השמירה
שאלות?	כן — דוחות, ביקורת, היסטוריה	לא

למה לא למחוק כשהמנה נאספה? כי מחר שואלים: "מה הזמנתי בחודש שעבר?". "כמה ביטולים היו ביום ג'?" · "הוכיחו שהזיכו הזה בוצע". הזמנה שהסתיימה היא עובדה עסקית — היא נשארת במערכת, מסוננת מהמסכים התפעוליים, בלי חצים עסקיים החוצה. זהו ארכיון: מצב אמיתי עם `entry / setStatus('ARCHIVED')` ובלי `do` — "להישמר" אינו התנהגות.

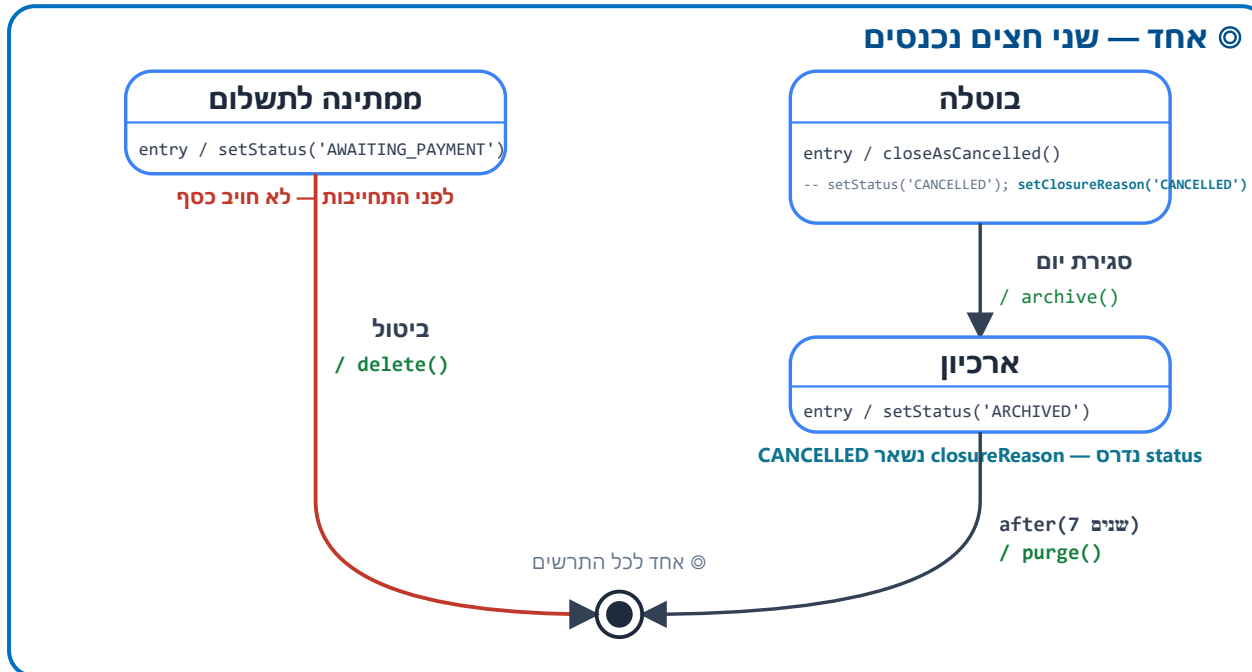
- מידע אישי
הקצר ביותר שניתן להגן עליו — פרטיות
- תפעולי
1-2 שנים — השוואה שנה-מול-שנה
- חוזי
7 שנים — חוק ההתיישנות
- כספי
7 שנים — הוראות ניהול ספרים (מס הכנסה)
- תקופת השמירה היא החלטה עם נימוק. (נימים-X) `after()` הוא מציין-מקום, לא החלטה — ייפסל. בוחרים שורה ומנמקים במשפט:

ההזמנה חייבה כסף ⇒ רשומה כספית ⇒ `purge()` / `after(7 שנים)`. הנימוק הוא חלק מהתרשים.

ארכיון או מחיקה? — מבחן העובדה העסקית

ארכיון הוא ברירת המחדל. מחיקה ישירה היא חריג — ורק מי שעובר את המבחן מקבל אותה

האם האובייקט הפך אי-פעם למשהו שדוח, ביקורת או אדם יוכלו לשאול עליו אחר כך?
כן $\Rightarrow$ מצב תוצאה $\leftarrow$ ארכיון $\leftarrow$ $\odot$ · לא $\Rightarrow$ `delete()` / אירוע ישירות ל- $\odot$ — אותו $\odot$ אחד



✗ לא הפכה לעובדה — מוחקים

הזמנה שננטשה לפני תשלום: הסטודנט מילא סל, הגיע למסך התשלום, וסגר את האפליקציה. לא חויב כסף, המטבח לא שמע עליה, אף דוח לא יספור אותה. מ-ממתינה לתשלום: `delete()` / ביטול ישר ל- $\odot$. לארכב אותה = לספור "ביטול" שמעולם לא קרה ולזהם את דוח הביטולים.

✓ עובדה עסקית — מארכבים

הזמנה ששולמה ובוטלה (ע"י הסטודנט לפני הכנה, או ע"י הצוות כשפריט אזל): הזיכוי הוא עובדה — כסף עבר חזרה, ומישהו ישאל עליו. בוטלה $\leftarrow$ `archive()` / סגירת יום $\leftarrow$ ארכיון, ורק `purge()` / `after(7 שנים)` מוחק.

`closureReason` נקבע במצב התוצאה, כי ה-`entry` של ארכיון דורס את `status`. בלי זה הארכיון יודע שנסגר אבל לא למה.

רק ממצב שלפני התחייבות. אחרי שכסף עבר, המטבח החל, או נשלחה התראה — האובייקט כבר עובדה, והדרך היחידה החוצה היא ארכיון.

אירוע אמיתי על חץ המחיקה — `delete()` / ביטול, לעולם לא `after(...)`. מחיקה ישירה קורית כי מישהו עשה משהו, לא כי עבר זמן.

⚠ ביטול אינו מחיקה

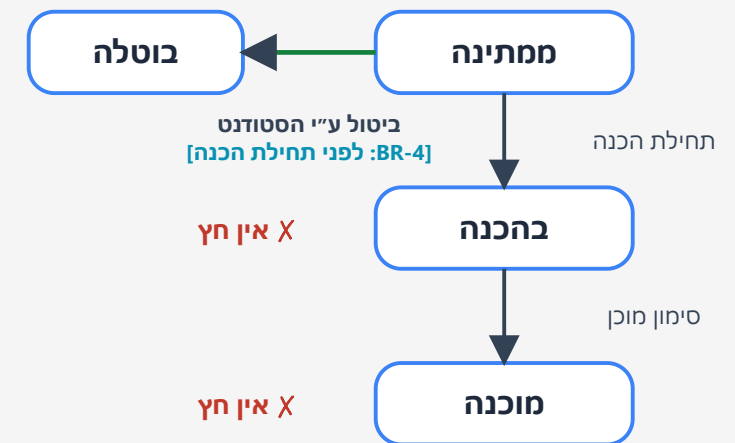
"הסטודנט ביטל, אז מוחקים" — לא. ביטול אחרי תשלום הוא עובדה שמדווחים עליה ומזכים בגינה $\Rightarrow$ בוטלה $\Rightarrow$ ארכיון. רק רשומה שמעולם לא הייתה תקפה — טיוטה נטושה, כפילות, שגיאת הקלדה — נמחקת.

חוק עסקי = חץ שחסר

תרשים המצבים אוסף חוקים מהקטלוג של שלב הניתוח — בחץ שאין, בתנאי שמפנה, ובפעולה שחסרה

BR-4 — החוק הוא החץ שלא מצויר

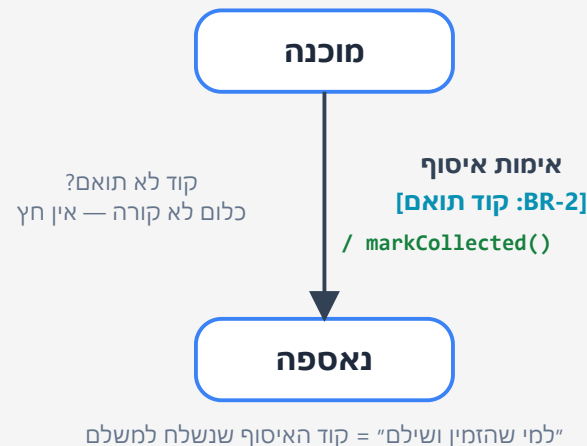
ביטול הזמנה מותר עד שהיא עוברת ל"בהכנה" — לא אחריה.



ההיעדר הוא החוק. אין "חץ אסור" ב-UML — מה שלא מצויר, לא קורה. הבדוק צריך לחפש מה חסר.

BR-2 — החוק הוא תנאי שמפנה לקטלוג

הזמנה נמסרת אך ורק למי שהזמין ושילם.



מפנים, לא משכתבים. [קוד תואם: BR-2] — החוק כתוב פעם אחת בקטלוג; כשהוא ישתנה, מוצאים לפי ההפניה את כל החצים שמושפעים.

BR-3 — חוק שהוכרע תוך כדי עיצוב: פעולה שחסרה

הזמנה ששולמה ולא נאספה עד סוף חלון האיסוף — מדיניות שטרם הוחלטה החיוב נשאר, ללא זיכוי.



החוק = פעולה שחסרה. שני המצבים נראים זהה — פעולת כניסה אחת כל אחד — וההבדל הוא בגוף: ב `closeAsCancelled()` יש `refund()`, ב `closeAsNotCollected()` — אין. עד היום זה היה חור; התרשים הכריח את הקמפוס להחליט — המעצב השיג את ההחלטה, לא ניחש אותה.

העיצוב לא יוצר בית שני לחוקים — הוא מפזר את האכיפה. כל חוק כתוב פעם אחת בקטלוג, וכל תרשים מפנה אליו:

מה ה-AI צריך מכם

רשימת חוקים חד-משמעית. הטייטה שלו ציירה "ביטול" מכל מצב — כי אף אחד לא כתב לו BR-4, והוא המציא מדיניות. חוק שלא מוצאים בתרשים הוא חוק שטרם הוחלט — סוגרים אותו בהחלטה של הבעלים, לא בניחוש (שלו או שלכם).

עיצוב

בונים תרשים מצבים — חמישה שלבים

1

האובייקט

של מי מחזור החיים?

- אובייקט אחד לתרשים — עם מחזור חיים וחוקים על "מה מותר מתי"
- איזו תכונה מחזיקה את המצב?
- שני ערכים? זה בוליאני, לא תרשים

בקמפוס: ההזמנה —
Order.status

שלכם

2

המצבים

מה האובייקט הוא לאורך זמן?

- שם תואר / בינוני — לא פועל ("בהכנה", לא "הכנה")
- מצב מוצדק בהתנהגות: אירועים אחרים או פעולות אחרות — לא בהיסטוריה
- תוצאות עסקיות הן מצבים, לא ©

בקמפוס: 8 מצבים + מצב-
על פעילה

מיוטה

3

אירועים ותנאים

מה מזיז אותו — ומתי מותר?

- כל חץ: / event [guard] action — חוץ מהחץ מ-•
- אותו אירוע, יעד שונה = תנאי; תנאי שנכשל = אין חץ
- חץ שחסר בכוונה = חוק עסקי

בקמפוס: BR-2 תנאי · BR-4
חץ חסר

מיוטה

4

פעולות

מה קורה בכניסה, ביציאה, בפנים?

- entry / setStatus(...) ■
בכל מצב, תמיד
- do רק למה שבאמת רץ ■
לאורך המצב
- כל פעולה = שיטה ■
שהמפתח יכתוב

בקמפוס: notifyStudent(),
refund()

מיוטה

5

בדיקה

זה תרשים מצבים — או תרשים זרימה?

- מעגלים, לולאות, +2 חצים ■
יוצאים — לא שרשרת מלמעלה למטה
- אחד, © אחד ■
- שני סופים: ארכיון ← ■
after(תקופה) / purge()
© ←

בקמפוס: 7 שנים — רשומה
כספית

שלכם

חלוקת העבודה

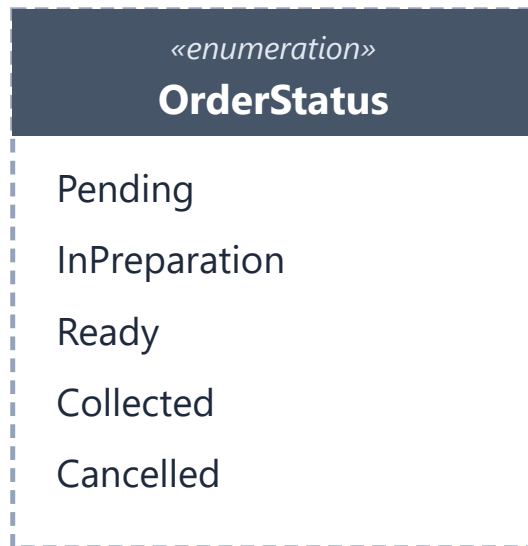
ה-AI מנסח את שלבים 2-4 בשניות מתוך תרחישי השימוש. שלב 1 (איזה אובייקט בכלל) ושלב 5 (הבדיקה) — שלכם. בלעדיהם תקבלו תרשים זרימה יפה ושגוי, בביטחון.

עיצוב

1 / 5 הזמנה בקמפוס — שלב 1: האובייקט

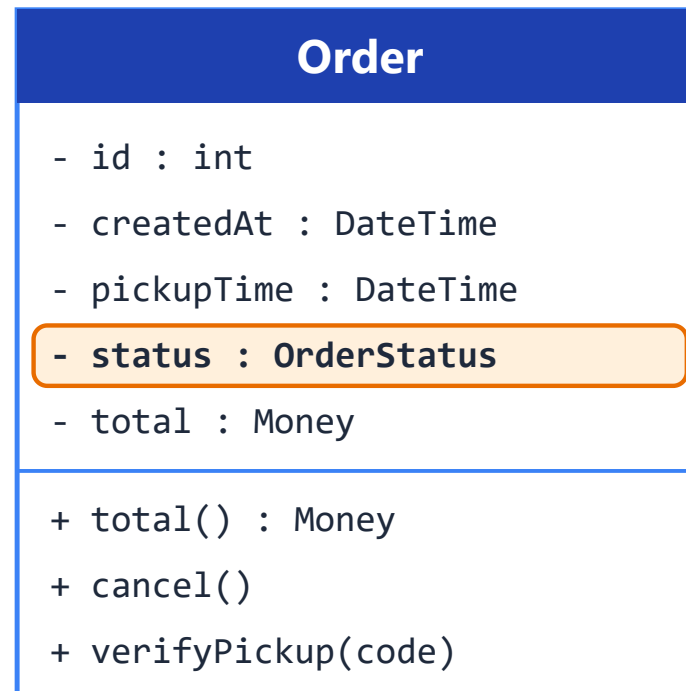
נקודות לדיון

1. האובייקט הוא **ההזמנה** — לא הסטודנט, לא פריט התפריט. למה דווקא היא?
2. **חמישה ערכים** — זה כל הסיפור? מה עם הזמנה שעוד **לא שולמה**? הזמנה **שלא נאספה**?
3. מי מעביר **ממתינה** ← **בהכנה**? הסטודנט? עובד המטבח? השעון?
4. הזמנה **שנאספה** — נעלמת מהמערכת? ומה עונים לסטודנט ששואל "מה הזמנתי בחודש שעבר?"



5 ערכים — זה כל הסיפור?

(אין קו בין enum למחלקה — טיפוס התכונה הוא הקישור)



ההזמנה — לא הסטודנט, לא פריט התפריט

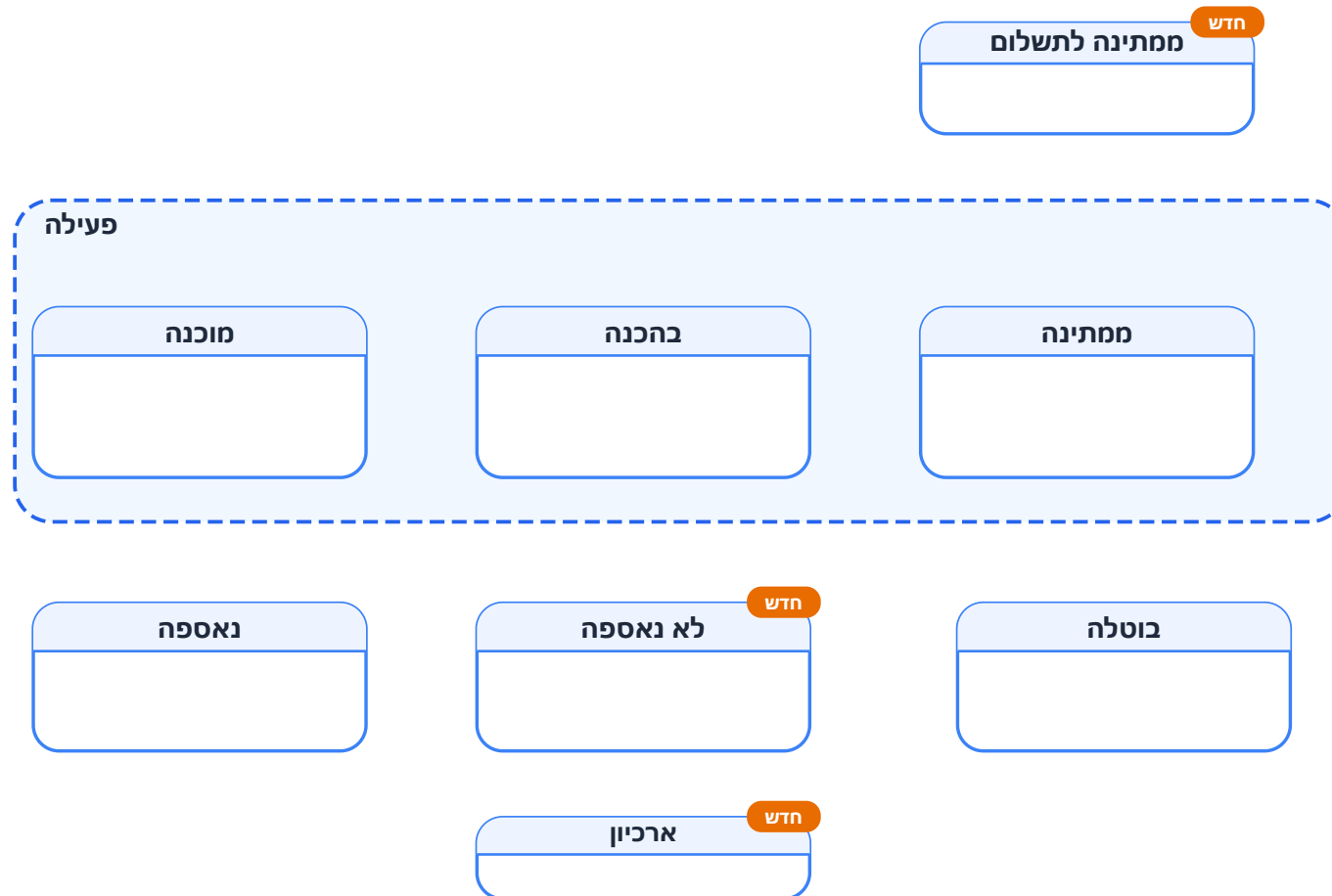
2 / 5 שלב 2: המצבים — מה ההזמנה היא לאורך זמן

נקודות לדיון

1. מצב = מה ההזמנה היא לאורך פרק זמן — לא מה קורה לה. שם תואר / בינוני, לא פועל.
2. "הכנה" — זה מצב? לא: זו פעילות. המצב הוא בהכנה; "תחילת הכנה" יהיה האירוע.
3. איזה מצב בא אחרי נאספה? ההזמנה עדיין קיימת בדוחות ← ארכיון.
4. מצב מוצדק בהתנהגות, לא בהיסטוריה: אם שני מצבים מקבלים אותם אירועים — מאחדים ושומרים תכונה.

חזור למודל המחלקות

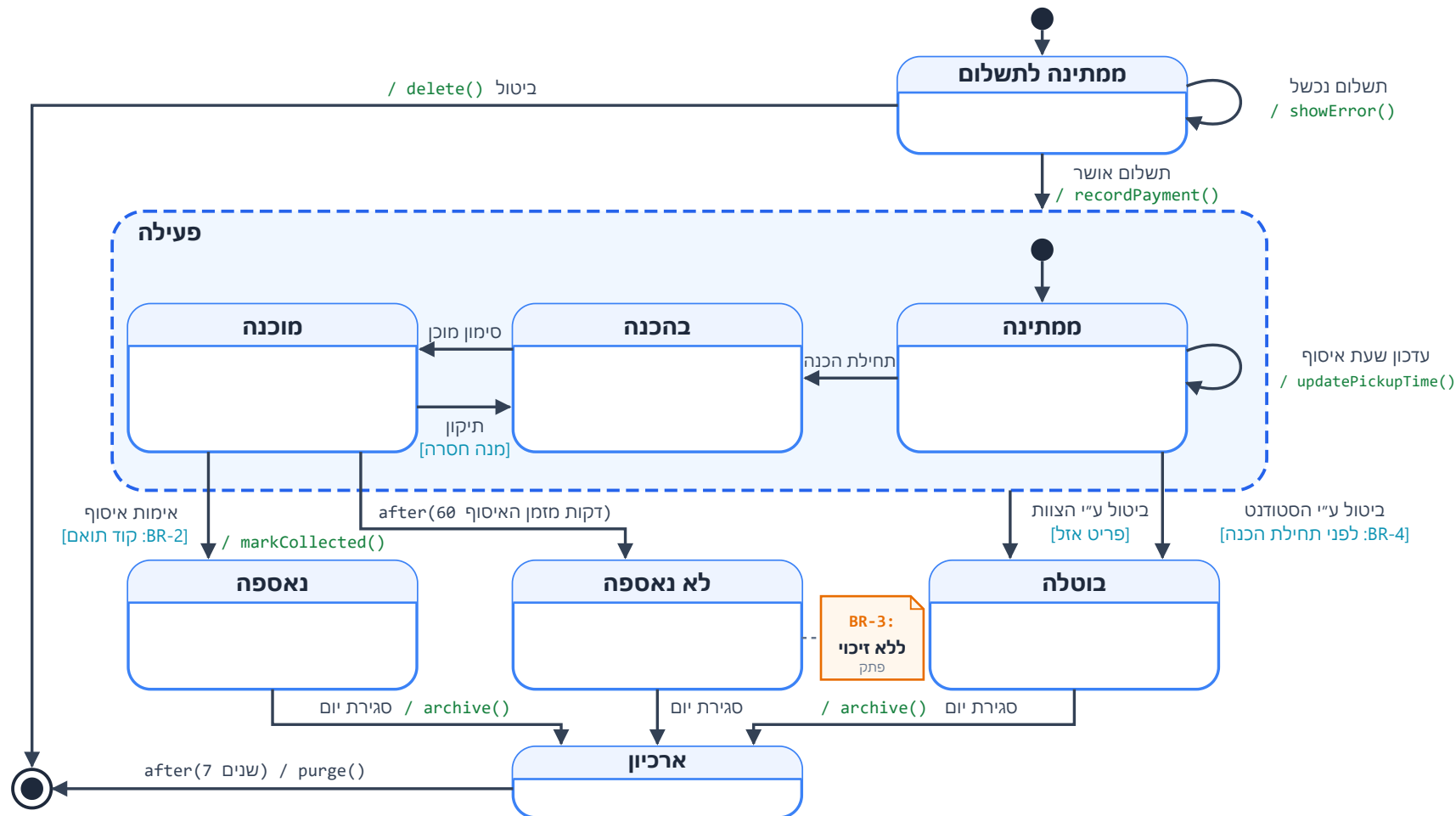
· `OrderStatus` מקבל: `AWAITING_PAYMENT` (למצב-העל)
`ACTIVE-I` `ARCHIVED` `NOT_COLLECTED`
· `Order` מקבל תכונה: `closureReason`



שלב 3: אירועים ותנאים — כל חץ הוא אירוע 3 / 5

נקודות לדיון

1. אותו אירוע, יעד שונה? אימות איסוף
[קוד תואם: BR-2] ← נאספה. קוד לא
תואם? לא קורה כלום — תנאי
שנכשל אינו חץ.
2. בוטלה יכולה לחזור לחיים? לא —
אין חץ יוצא ממנה חזרה. חץ שאין =
מעבר אסור.
3. המטבח גילה שפריט אזל? ביטול ע"י
הצוות — חץ אחד מגבול פעילה
במקום שלושה זהים.
4. מעגל מוכנה ↔ בהכנה (תיקון), שתי
לולאות עצמיות, שלוש דרכים
החוצה — זה לא תרשים זרימה.
5. ממתינה לתשלום ← © ישירות:
הזמנה שלא שולמה מעולם לא
הייתה עובדה עסקית. מחיקה, לא
ארכיון.



כל חץ = אירוע (עם תנאי בסוגריים). אין חץ = אסור. חץ שחסר בכוונה = חוק עסקי מהקטלוג.



```

תשלום אושר + recordPayment()
תחילת הכנה + startPreparation()
סימון מוכן + markReady()
אימות איסוף + verifyPickup(code)
[BR-2]
תיקון + adjust()
ביטול [BR-4] + cancel()
עדכון שעת איסוף + updatePickupTime(t)
סגירת יום + archive()
— after(60 דק' טיימר) + markNotCollected()
— after(7 שנים) + purge()

```

entry ← פעולה אחת; צריך יותר? שיטת עטיפה (פרטית)

```

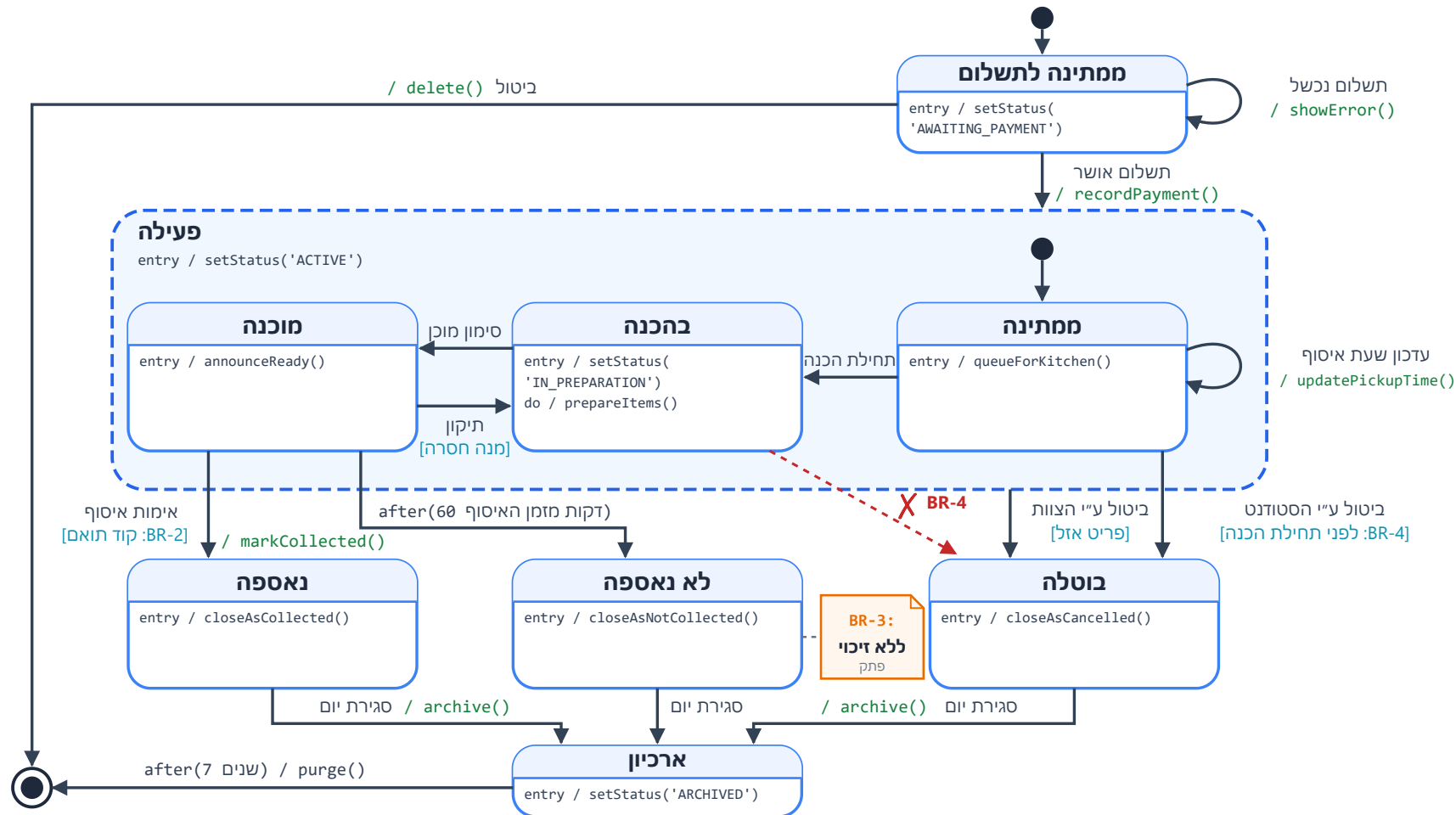
        setStatus + - queueForKitchen()
    notifyKitchen
        setStatus + - announceReady()
    notifyStudent
        setStatus + - closeAsCollected()
    setClosureReason
refund() גם - closeAsCancelled()
refund [BR- ללא - closeAsNotCollected()
3]
do — רץ לאורך המצב - prepareItems()

```

התראה נשלחת פעם אחת בכניסה

אירוע ← שיטה ציבורית · entry ← שיטה אחת, ובה setStatus · שתי פעולות בכניסה = שיטת עטיפה.

5 / 5 שלב 5: בדיקה — התרשים המוגמר מול הרשימה



שלב 5: הבדיקה

✓ לא תרשים זרימה: מעגל מוכנה → בהכנה, שתי לולאות עצמיות, שלוש יציאות, חצים לכל כיוון

✓ אחד, אחד, אחד — ואילו שני חצים נכנסים

✓ לכל חץ אירוע — חוץ מהחץ מ-•

✓ תוצאות הן מצבים: נאספה / בוטלה / לא נאספה — לא ©

✓ זנב ארכיון עם תקופה קונקרטית: (שנים 7) after — רשומה כספית

✓ BR-4 = החץ שחסר: אין ביטול-סטודנט מבהכנה (ולא ממוכנה)

✓ BR-3 — הוכרע: ללא זיכוי, ולכן אין refund() בתוך closeAsNotCollected() — חוק כפעולה שחסרה

מחזור החיים של ההזמנה — התרשים החי

הזמנה בקמפוס: 8 מצבים, מצב-על אחד, • אחד, © אחד — וחוק אחד שהוכרע תוך כדי העיצוב

מה לראות בתרשים

 מצב-על פעילה — ביטול ע"י הצוות יוצא מהגבול: חץ אחד במקום שלושה

 המעגל מוכנה ↔ בהכנה — תיקון
[מנה חסרה] הוא חץ אחורה: זה לא תרשים זרימה

 שתי לולאות-עצמיות — תשלום נכשל ·
עדכון שעת איסוף: הנתון משתנה, המצב לא

 סוף אחד, שני חצים — זנב הארכיון
purge() / (שנים 7) after לעומת המעקף
delete() / ביטול מממתינה לתשלום

 תנאים שמפנים לקטלוג —
[קוד תואם: BR-2] · [BR-4]; BR-4-I הוא גם
החץ שחסר מבהכנה

 BR-3 — הוכרע: ללא זיכוי. הכלל נראה
כפעולה שחסרה — אין refund() בלא
נאספה; הפתק רק מסביר

 setStatus בכל מצב — גם פעילה וגם
ארכיון; closureReason שורד את הארכיון

התרשים חי

לחצו על מצב או על חץ — כל אחד מסביר את עצמו

תרשים מצבים — מחזור החיים של הזמנה בקמפי

לחצו על מצב או על חץ לקבלת הסבר

Download .xmi (Visual Paradigm)

עיצוב

מהתרשים למימוש: טבלת מעברים

איך קוראים את הטבלה

שורה = המצב הנוכחי · עמודה =
האירוע שהגיע · התא = המצב הבא.
תא ריק (—) = האירוע לא חוקי במצב
הזה. המערכת אוכפת: הכפתור נעול, או
שהשיטה זורקת חריגה.

התא הכתום = BR-4. החוק "ביטול
רק לפני תחילת הכנה" הוא לא קוד
מיוחד — הוא תא ריק בטבלה.

⊙ בשני תאים בלבד: מחיקה ישירה לפני
תשלום (לא הייתה עובדה עסקית),
ו-purge() אחרי 7 שנים מהארכיון.

הטבלה היא המפרט המדויק לקוד
(השקף הבא) — או ל-check-
constraint במסד הנתונים.

לא בעמודות, כדי שיהיה קריא: שתי הלולאות
העצמיות (תשלום נכשל, עדכון שעת איסוף)
ותיקון (מוכנה ← בהכנה).

מצב \ אירוע	תשלום אושר	תחילת הכנה	סימון מוכן	אימות איסוף	ביטול (סטודנט)	ביטול (צוות)	סגירת יום	after(...)
ממתינה לתשלום	ממתינה פעילה	—	—	—	⊙ delete()	—	—	—
ממתינה	—	בהכנה	—	—	בוטלה [BR-4]	בוטלה	—	—
בהכנה	—	—	מוכנה	—	— BR-4	בוטלה	—	—
מוכנה	—	—	—	נאספה [BR-2]	— BR-4	בוטלה	—	לא נאספה 60 דק'
נאספה	—	—	—	—	—	—	ארכיון	—
בוטלה	—	—	—	—	—	—	ארכיון	—
לא נאספה BR-3: ללא זיכוי	—	—	—	—	—	—	ארכיון	—
ארכיון	—	—	—	—	—	—	—	⊙ 7 שנים · purge()

אותו תרשים, כטבלה: כל תא מלא הוא חץ, וכל תא ריק הוא חוק.

מהתרשים לקוד

תרשים המצבים הוא המפרט שה-AI אוסף — בלי תרשים, הוא מנחש את החוקים

החוק BR-4 — החץ שחסר בתרשים — כקוד (C#):

```
enum OrderStatus { AwaitingPayment, Pending, InPreparation, Ready,
    Collected, Cancelled, NotCollected, Archived }

public void Cancel() // event
{
    if (Status != OrderStatus.Pending) // guard: BR-4
        throw new InvalidOperationException("BR-4");
    CloseAsCancelled(); // the state's ONE entry action
}

private void CloseAsCancelled() // entry / closeAsCancelled()
{
    Status = OrderStatus.Cancelled; // setStatus
    ClosureReason = "CANCELLED";
    Refund();
}
```

🌐 לא תלוי שפה

המיפוי הוא UML ← כל שפה: ב-Python או Java אותם ארבעה חלקים — enum, שיטה, if, השמה. #C רק כי פרויקט הדוגמה כתוב בה.

עיצוב

מה הופך למה

בתרשים	בקוד
מצב	enum-ב — <code>OrderStatus.Pending</code>
אירוע	שיטה ציבורית — <code>public void Cancel()</code>
תנאי [guard]	if שזורק חריגה כשהוא נכשל
מעבר	השמה — <code>Status = OrderStatus.Cancelled</code>
exit / entry	שיטה אחת שנוקראת בכניסה (העטיפה), ובגופה ההשמה ל-Status והשאר
after(...) · do	משימה שרצה ברקע · משימה מתוזמנת (טיימר)

🤖 התרשים הוא המפרט

בהרצאת המימוש ה-AI מייצר בדיוק את זה מהתרשים: enum, שיטה לכל אירוע, if לכל תנאי. בלי תרשים — הוא מנחש, ומבטל הזמנה שכבר בהכנה, בביטחון. הצוות עדיין אחראי למה שאין בתרשים: הרשאות, טיימרים בייצור, שער התשלום.

חידון: מה לא בסדר בתרשים?

שישה קטעים מטיוטה של AI — בכל אחד טעות אחת (לפחות). לחצו על כרטיס כדי להפוך אותו

קודם מחליטים בכיתה — אחר כך הופכים. השאלה לכל כרטיס: איזה כלל נשבר, ומה התיקון?

קזס הזיז האצזח — © הזזזז

חִיקְסָשׁ הַאֲצוּחוֹ — "חִיקְסָשׁ אֶל אֶבֶן הַמּוֹשָׁחָה" וְשׁוּרִים ©: לְלֶחֶם
© (חֹמֶה) .מִצְעוֹן נֶחֱם הַסָּמָה אֶל \ הַסָּטוֹ \ הַסָּמָה. חֹד הַזֶּה
(הַמִּלְאָה לְיִשְׂרָאֵל חֹד אֵיחָ הַמּוֹשָׁחָה — JMU-בִּרְחוֹם)

© → (מזי חתום \ archive()) ויטרא → נתשולש: וקיתח
 6.תפר (ז מזש) \ epg() מ שחא

סיכום סיכום שלי, האניסיסי סיכום זה

יאגזח צחה לט חצווי חטלחה. אלאך יאו מיזיטו יא :ללח
ללח צח וזיא — לשטש יאחו. [שטש]

דוק: [S-98] מוסיף חומרים: הפסאג → הגלום זחא ין :ןקיהר
מילכ החורז אל ⇒ מאור אל דוק. המלאכותי (b) \ ([מאור]

שזרמא ילם חץ

המושג נחמד אך לא. •-אם נסתכל על חזיונות ילד דוֹחִיחַ נסתכל: לֵאמֹר שוֹחִים אֵלַי, רִשְׁמוּ אֵלַי אֶת הַחֶמֶד — "חֶמֶד חֶמֶד" לֵאמֹר

דבש) המזה מזיקה: שוריא אצונו המזחם → המיחמוס; וקייחה
(ממנחה חא חק'ל חסטמה).

רשימת חסמאש פן

קוּחַ. הַנִּצְחָה חֲלִיחָה דֵּשׁ קָרָא לְטַבּוֹטָא שְׁטוּטָא — א-ת-ח
אֵל טוֹשֵׁה חֲרָמָה: **סִסְחָץ** אִזְּהוּ מִיבָצָה מִשְׁחָרָה יִקְסָא
הִנֵּה צֶמֶד "לְוִיטִי" חֲשִׁיצָה.

[illegible]

שזרזא אף, חזרזש איה "חלסח"

זיס מאכטן "המטה". חרוקש גאז — **אויזא** מיטמוץ יץ לא: **ללאח**
צום און אלעמא צעמו — וואס **איה** המומחה חנה נאמט; חרוקד
חסח.

[illegible]

רסמ זמאש X-2, ר'ער פולער: מחזקט יחש

חחא פאש רחויח לאל דחא ל' — חחא \ fix \ ob — פתח \ תחת: ל'אלה
 חאזקאש חפישט חטיש? פייחש פירש; חחא חאושט פשו, פאפא
 וייצט אל, פיסט פטחחא איה חריטש חפוקחו. וחיחשל
 פוקט.

חשוב פתור \ archiver0nברר(): **נוקיה**

פותר: ז' יחיו, zutst2t9z('EDVIA') ; AgoIrchv(נ)

ז' חרידש רוח ⇒ **ריסס חושך — מישׁ** \ טרגפ()
פיץ.

אירוע על כל חץ · תנאי ולא מעוין · תוצאה היא מצב, לא © · פעילות היא do · חוק = חץ שחסר.

עיצוב

Artificial Intelligence = AI · שפת מידול אחידה · Unified Modeling Language = UML · בינה מלאכותית

תרגיל: קריאת תחזוקה במעונות

אותו תרחיש מהרצאת המחלקות — הפעם מחזור החיים של אובייקט אחד. ה-AI מצייר, ואתם מבקרים

אובייקט: קריאת תחזוקה במערכת המעונות

דייר פותח קריאה. משרד המעונות משבץ לה טכנאי. הטכנאי יכול להשהות אותה — ממתינה לחלק חילוף — ולחזור לטפל כשהחלק הגיע. בסיום, הדייר מאשר את התיקון — או פותח מחדש בתוך 7 ימים. המשרד רשאי לדחות קריאה כפולה של אותה תקלה.

1

הזינו ל-AI

תנו את התרחיש ל-Claude ובקשו **תרשים מצבים** של קריאת תחזוקה + רשימת **ההנחות** שהניח. אפשר ישירות בכלי `uml_state_diagram` של `sad-mcp` (התקנתם בהרצאה 1). בקשו גם: מהי תקופת השמירה בארכיון, ולמה.

2

בקרו לפי רשימת הבדיקה

- אחד; © רק היכן שהרשומה נמחקת (אחד מספיק) — ואירוע על **כל** חץ אחר; תנאים ב- []
- תוצאות (נסגרה / נדחתה) הן **מצבים** ← ארכיון ← `after(...)` עם תקופה **מנומקת**
- מחיקה ישירה ל-© **רק** לכפילות — היא מעולם לא הייתה עובדה
- מבחן ההתנהגות: "בטיפול" ו"ממתינה לחלק" — אותם אירועים? אם לא, **שני מצבים**
- פתיחה מחדש = **חץ חוזר** אחורה, עם התנאי [עד 7 ימים]

3

תקנו ונמקו

הציעו תרשים מתוקן, והסבירו **כל** שינוי — מה ה-AI פספס, המציא, או הניח, ולמה התיקון נכון. כל מצב חדש שהוספתם — מה ההתנהגות שמצדיקה אותו? וכל חץ שמחקתם — איזה חוק עסקי הוא מייצג?

⚠ המלכודות שנחפש

"שיבוץ" הוא אירוע — "טיפול" הוא פעילות (/ do) · קריאה כפולה ⇐ `delete()` ישירות, קריאה שנסגרה ⇐ ארכיון · "ממתינה לחלק" מקבלת אירוע אחר (החלק הגיע) ⇐ מצב נפרד · פתיחה מחדש היא מחזור, לא מצב חדש · © אחד, לא שניים.

לא מדרגים כמה מהר ה-AI צייר — מדרגים כמה חדה הביקורת שלכם.

עיצוב

מ-status שהוא רק ערך — למחזור חיים שמכתיב מה חוקי

status הוא רק ערך

מצבים · מעברים · אירועים · פעולות

מצבי-על ושני הסופים

בונים ובודקים

שישה עקרונות שנשארים איתכם

- מצב הוא מה שהאובייקט הוא לפרק זמן — ומצדיקה אותו התנהגות שונה, לא היסטוריה.
- על כל חץ אירוע; התנאי מכריע, הפעולה באה אחריו — / [guard] event action. תנאי שנכשל אינו חץ.
- • אחד, @ בסוף — ו-@ פירושו שהרשומה איננה, לא "ההזמנה הסתיימה" UML מתיר כמה @; אנחנו מציירים אחד).
- תוצאות הן מצבים ← ארכיון ← (תקופה) after עם בסיס: רשומה כספית ⇐ 7 שנים.
- חוק עסקי הוא חץ שחסר (BR-4); חוק שטרם הוחלט הוא חור — והתרשים הוא מה שמכריח להכריע (BR-3: ללא זיכוי).
- זה לא תרשים זרימה: מחזורים, לולאות עצמיות, כמה יציאות מכל מצב — ואף כן/לא.

מה תרשים המצבים החזיר לנו

↩ ה-enum גדל: ARCHIVED, NOT_COLLECTED, AWAITING_PAYMENT — ותכונה חדשה closureReason. תרשים המחלקות מתעדכן.

↩ רשימת השיטות של Order: recordPayment()
· announceReady() · prepareItems() · queueForKitchen()
· archive() · closeAsCancelled() · markCollected()
purge() — כל פעולה בתרשים היא שיטה, ופעולת כניסה שעושה שני דברים היא שיטת עטיפה אחת.

↩ הכרעה שהתרשים כפה: BR-3 — הזמנה ששולמה ולא נאספה: ללא זיכוי. פתוח מאז הרצאה 3, ונראה בתרשים כ- refund() שחסר.

מהעיצוב אל ההחלטה והמערכת הרצה: חקר ישימות, ובניית המערכת עם Claude Code.

הבא

אותו תהליך שעשינו בכיתה — על אובייקט שלא הרצנו

אובייקט: הזמנת חדר במערכת של מלון

אורח יוצר הזמנה חדשה לתאריכים; הקבלה מאשרת אותה רק אם יש חדר פנוי — אחרת דוחה. עד יומיים לפני ההגעה ההזמנה ניתנת לביטול ע"י האורח; ב-צ'ק-אין היא הופכת לשהות פעילה, וב-צ'ק-אאוט — רק אחרי שהחשבון שולם — היא מסתיימת. אורח שלא הגיע עד חצות $\Leftarrow$ אי-הגעה, וחייב לילה ראשון. תשלום שנכשל בעת האישור מבטל את ההזמנה.

1

הזינו ל-AI

תנו את התרחיש ל-Claude (או לכלי `uml_state_diagram`) ובקשו תרשים מצבים של הזמנת חדר + רשימת ההנחות שהניח.

2

בקרו לפי רשימת הבדיקה

- אחד, © רק למחיקת הרשומה, אירוע על כל חץ, תנאים ב- [ן · תוצאות (הושלמה / בוטלה) הן מצבים, ואחריהן ארכיון · מבחן ההתנהגות: האם "ממתינה לתשלום" היא מצב — אילו אירועים היא מקבלת שאחרים לא? · ומצאו את מלכודות תרשים הזרימה: "חדר פנוי?" עם כן/לא, "טיפול בהזמנה" על חץ, © אחרי כל תוצאה.

3

תקנו ונמקו

תרשים מתוקן עם זנב ארכיון ו- (תקופה) `after` מנומקת — למה דווקא המספר הזה? הסבירו כל שינוי: מה ה-AI פספס, המציא, או הניח.

מה מגישים

- הפלט המקורי של ה-AI (התרשים וההנחות) + הפרומפט ששלחתם.
- טבלת ביקורת: לכל ליקוי — מה הבעיה, לאיזה כלל היא נוגעת, והתיקון.
- התרשים המתוקן שלכם (מהכלי או ביד), ופסקה קצרה: מה ה-AI דווקא עשה טוב.

לא מדרגים כמה יפה ה-AI צייר — מדרגים כמה הייתה הביקורת שלכם.

THE END

OR THE BEGINNING...

תודה!

הבא — מהעיצוב אל ההחלטה והמערכת הרצה:
חקר ישימות, ובניית המערכת עם Claude Code

ניתוח ועיצוב מערכות מידע · הרצאה 8 — תרשים מצבים State Diagram